Cryptography & Network Security Lab

List of Experiments with Programs

1	Write a C program that contains a string (char pointer) with a value ‘Hello world’. The program should XOR each character in this string with 0 and displays the result.
2	Write a C program that contains a string (char pointer) with a value ‘Hello world’. The program should AND or and XOR each character in this string with 127 and display the result.
3	Write a Java program to perform encryption and decryption using the following algorithms: a. Ceaser cipher b. Substitution cipher c. Hill Cipher
4	Write a C/JAVA program to implement the DES algorithm logic.
5	Write a C/JAVA program to implement the Blowfish algorithm logic.
6	Write the RC4 logic in Java Using Java cryptography; encrypt the text “Hello world” using Blowfish. Create your own key using Java key tool.
7	Write a Java program to implement RSA algorithm.
8	Implement the Diffie-Hellman Key Exchange mechanism.
9	Calculate the message digest of a text using the SHA-1 algorithm in JAVA.
10	Calculate the message digest of a text using the MD5 algorithm in JAVA.

Experiment 1

Question:

Write a C program that contains a string (char pointer) with a value ‘Hello world’. The program should XOR each character in this string with 0 and displays the result.

Code (exp1.c):

```
#include <stdio.h>

int main()
{
    const char str[] = "Hello World";
    char res[100];
    int i = 0;

    printf("Input string is: %s\n", str);

    while (str[i] != '\0')
    {
        res[i] = str[i] ^ 0;
        i++;
    }

    res[i] = '\0';

    printf("Exclusive XOR: %s\n", res);

    return 0;
}
```

Output:

```
Input string is: Hello World
Exclusive XOR: Hello World
```

Experiment 2

Question:

Write a C program that contains a string (char pointer) with a value ‘Hello world’. The program should AND or and XOR each character in this string with 127 and display the result.

Code (exp2.c):

```
#include <stdio.h>

int main()
{
    const char *str = "Hello World";
    int i = 0;

    printf("Original String: %s\n", str);

    printf("\nAND with 127:\n");

    while (str[i] != '\0')
    {
        printf("%c -> %c (ASCII: %d -> %d)\n",
               str[i],
               str[i] & 127,
               str[i],
               str[i] & 127);
        i++;
    }

    i = 0;

    printf("\nXOR with 127:\n");

    while (str[i] != '\0')
    {
        printf("%c -> %c (ASCII: %d -> %d)\n",
               str[i],
               str[i] ^ 127,
               str[i],
               str[i] ^ 127);
        i++;
    }

    return 0;
}
```

Output:

```
Original String: Hello World

AND with 127:
H -> H (ASCII: 72 -> 72)
e -> e (ASCII: 101 -> 101)
l -> l (ASCII: 108 -> 108)
l -> l (ASCII: 108 -> 108)
o -> o (ASCII: 111 -> 111)
  ->   (ASCII: 32 -> 32)
W -> W (ASCII: 87 -> 87)
o -> o (ASCII: 111 -> 111)
r -> r (ASCII: 114 -> 114)
l -> l (ASCII: 108 -> 108)
d -> d (ASCII: 100 -> 100)

XOR with 127:
H -> 7 (ASCII: 72 -> 55)
e -> ^Z (ASCII: 101 -> 26)
l -> ^S (ASCII: 108 -> 19)
l -> ^S (ASCII: 108 -> 19)
o -> ^P (ASCII: 111 -> 16)
  -> _ (ASCII: 32 -> 95)
W -> ( (ASCII: 87 -> 40)
o -> ^P (ASCII: 111 -> 16)
r ->
 (ASCII: 114 -> 13)
l -> ^S (ASCII: 108 -> 19)
d -> ^[ (ASCII: 100 -> 27)
```

Experiment 3

Question:

Write a Java program to perform encryption and decryption using the following algorithms:
a. Ceaser cipher b. Substitution cipher c. Hill Cipher

3(a) Caesar Cipher

Code (exp3a.java):

```
import java.util.*;

public class exp3a
{
    public static String encrypt(String text, int shift)
    {
        StringBuilder res = new StringBuilder();

        for (char ch : text.toCharArray())
        {
            if (Character.isLetter(ch))
            {
                char base = Character.isUpperCase(ch) ? 'A' : 'a';
                char ency = (char) (base + (ch - base + shift) % 26);
                res.append(ency);
            }
            else
            {
                res.append(ch);
            }
        }

        return res.toString();
    }

    public static String decrypt(String text, int shift)
    {
        return encrypt(text, 26 - (shift % 26));
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter plain text:");
        String plainText = sc.nextLine();

        System.out.println("Enter shift key (0-25):");
        int shift = sc.nextInt();

        String encrypted = encrypt(plainText, shift);
        System.out.println("Encrypted text: " + encrypted);

        String decrypted = decrypt(encrypted, shift);
        System.out.println("Decrypted text: " + decrypted);

        sc.close();
    }
}
//javac exp3a.java
// java exp3a
```

Output:

```
Enter plain text: Hello World
Enter shift key (0-25): 3
Encrypted text: Khoor Zruog
Decrypted text: Hello World
```

3(b) Substitution Cipher

Code (exp3b.java):

```
import java.util.*;

public class exp3b
{
    static final String ALPHABET =
        "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

    static final String SUBSTITUTION_KEY =
        "QWERTYUIOPASDFGHJKLZXCVBNM";

    public static String encrypt(String plainText)
    {
        StringBuilder encrypted = new StringBuilder();

        for (char c : plainText.toUpperCase().toCharArray())
        {
            int idx = ALPHABET.indexOf(c);

            if (idx != -1)
            {
                encrypted.append(SUBSTITUTION_KEY.charAt(idx));
            }
            else
            {
                encrypted.append(c);
            }
        }

        return encrypted.toString();
    }

    public static String decrypt(String cipherText)
    {
        StringBuilder decrypted = new StringBuilder();

        for (char c : cipherText.toUpperCase().toCharArray())
        {
            int idx = SUBSTITUTION_KEY.indexOf(c);

            if (idx != -1)
            {
                decrypted.append(ALPHABET.charAt(idx));
            }
            else
            {
                decrypted.append(c);
            }
        }

        return decrypted.toString();
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter plain text: ");
        String plainText = sc.nextLine();

        String encryptText = encrypt(plainText);

        System.out.println("Encrypted text: " + encryptText);

        String decryptText = decrypt(encryptText);

        System.out.println("Decrypted text: " + decryptText);
        sc.close();
```

```
    }
}
//javac exp3b.java
//java exp3b
```

Output:

```
Enter plain text: Hello World
 Encrypted text: ITSSG VGKSR
Decrypted text: HELLO WORLD
```

3(c) Hill Cipher

Code (exp3c.java):

```
import java.util.*;

public class exp3c
{
    static int[][] keyMatrix = new int[2][2];
    static int[][] inverseKeyMatrix = new int[2][2];

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter 4 integers for 2x2 key matrix:");

        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
                keyMatrix[i][j] = sc.nextInt();
            }
        }

        if (!findInverseKey())
        {
            System.out.println("Key Matrix is not invertible. Exiting.");
            return;
        }

        sc.nextLine();

        System.out.println("Enter plaintext (uppercase only):");
        String plaintext = sc.nextLine()
                             .toUpperCase()
                             .replaceAll("[^A-Z]", "");

        if (plaintext.length() % 2 != 0)
        {
            plaintext += "X";
        }

        String encrypted = encrypt(plaintext);
        System.out.println("Encrypted Text: " + encrypted);

        String decrypted = decrypt(encrypted);
        System.out.println("Decrypted Text: " + decrypted);

        sc.close();
    }

    public static String encrypt(String text)
    {
        StringBuilder result = new StringBuilder();

        for (int i = 0; i < text.length(); i += 2)
        {
            int[] pair = {
                text.charAt(i) - 'A',
                text.charAt(i + 1) - 'A'
            };

            int[] encryptedPair = new int[2];

            for (int row = 0; row < 2; row++)
            {
                encryptedPair[row] =
                    (keyMatrix[row][0] * pair[0]
                    + keyMatrix[row][1] * pair[1]) % 26;
            }
```

```
            result.append((char)(encryptedPair[0] + 'A'));
            result.append((char)(encryptedPair[1] + 'A'));
        }

        return result.toString();
    }

    public static String decrypt(String text)
    {
        StringBuilder result = new StringBuilder();

        for (int i = 0; i < text.length(); i += 2)
        {
            int[] pair = {
                text.charAt(i) - 'A',
                text.charAt(i + 1) - 'A'
            };

            int[] decryptedPair = new int[2];

            for (int row = 0; row < 2; row++)
            {
                decryptedPair[row] =
                    (inverseKeyMatrix[row][0] * pair[0]
                    + inverseKeyMatrix[row][1] * pair[1]) % 26;

                if (decryptedPair[row] < 0)
                {
                    decryptedPair[row] += 26;
                }
            }

            result.append((char)(decryptedPair[0] + 'A'));
            result.append((char)(decryptedPair[1] + 'A'));
        }

        return result.toString();
    }

    public static boolean findInverseKey()
    {
        int a = keyMatrix[0][0];
        int b = keyMatrix[0][1];
        int c = keyMatrix[1][0];
        int d = keyMatrix[1][1];

        int det = (a * d - b * c) % 26;

        if (det < 0)
        {
            det += 26;
        }

        int detInv = -1;

        for (int i = 0; i < 26; i++)
        {
            if ((det * i) % 26 == 1)
            {
                detInv = i;
                break;
            }
        }

        if (detInv == -1)
        {
            return false;
        }

        inverseKeyMatrix[0][0] = (d * detInv) % 26;
```

```
        inverseKeyMatrix[0][1] = (-b * detInv) % 26;
        inverseKeyMatrix[1][0] = (-c * detInv) % 26;
        inverseKeyMatrix[1][1] = (a * detInv) % 26;

        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
                if (inverseKeyMatrix[i][j] < 0)
                {
                    inverseKeyMatrix[i][j] += 26;
                }
            }
        }

        return true;
    }
}
//javac exp3c.java
//java exp3c
```

Output:

```
Enter 4 integers for 2x2 key matrix:
3 3 2 5
Enter plaintext (uppercase only):
HELLO
Encrypted Text: HIOZHN
Decrypted Text: HELLOX
```

Experiment 4

Question:

Write a C/JAVA program to implement the DES algorithm logic.

Code (exp4.java):

```
import java.util.Base64;
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;

public class exp4
{
    public static SecretKey generateKey() throws Exception
    {
        KeyGenerator keyGenerator = KeyGenerator.getInstance("DES");
        return keyGenerator.generateKey();
    }

    public static String encrypt(String plainText, SecretKey secretKey)
            throws Exception
    {
        Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");
        cipher.init(Cipher.ENCRYPT_MODE, secretKey);

        byte[] encryptedBytes = cipher.doFinal(plainText.getBytes("UTF-8"));

        return Base64.getEncoder().encodeToString(encryptedBytes);
    }

    public static String decrypt(String cipherText, SecretKey secretKey)
            throws Exception
    {
        Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");
        cipher.init(Cipher.DECRYPT_MODE, secretKey);

        byte[] decryptedBytes = cipher.doFinal(
                Base64.getDecoder().decode(cipherText)
        );

        return new String(decryptedBytes, "UTF-8");
    }

    public static void main(String[] args)
    {
        try
        {
            String plainText = "Hello, DES Encryption";

            System.out.println("Plain Text: " + plainText);

            SecretKey key = generateKey();

            String encryptedText = encrypt(plainText, key);

            System.out.println("Encrypted Text: " + encryptedText);

            String decryptedText = decrypt(encryptedText, key);

            System.out.println("Decrypted Text: " + decryptedText);
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
    }
```

```
}
//javac exp4.java java exp4
```

Output:

```
Plain Text: Hello, DES Encryption
Encrypted Text: otO5u+G/vrLPBUnyOp0pTDBephJ4D2G4
Decrypted Text: Hello, DES Encryption
```

Experiment 5

Question:

Write a C/JAVA program to implement the Blowfish algorithm logic.

Code (exp5.java):

```
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;

public class exp5
{
    public static String encrypt(String plainText, SecretKey key)
            throws Exception
    {
        Cipher cipher = Cipher.getInstance("Blowfish");
        cipher.init(Cipher.ENCRYPT_MODE, key);

        byte[] encryptedBytes =
                cipher.doFinal(plainText.getBytes(StandardCharsets.UTF_8));

        return Base64.getEncoder().encodeToString(encryptedBytes);
    }

    public static String decrypt(String cipherText, SecretKey key)
            throws Exception
    {
        Cipher cipher = Cipher.getInstance("Blowfish");
        cipher.init(Cipher.DECRYPT_MODE, key);

        byte[] decryptedBytes =
                cipher.doFinal(Base64.getDecoder().decode(cipherText));

        return new String(decryptedBytes, StandardCharsets.UTF_8);
    }

    public static SecretKey generateBlowfishKey(int keySize)
            throws Exception
    {
        KeyGenerator keyGenerator =
                KeyGenerator.getInstance("Blowfish");

        keyGenerator.init(keySize);

        return keyGenerator.generateKey();
    }

    public static void main(String[] args)
    {
        try
        {
            SecretKey key = generateBlowfishKey(128);

            String plainText = "Hello, Blowfish Encryption";

            System.out.println("Plain Text: " + plainText);

            String encryptedText = encrypt(plainText, key);

            System.out.println("Encrypted Text: " + encryptedText);

            String decryptedText = decrypt(encryptedText, key);

            System.out.println("Decrypted Text: " + decryptedText);
```

```
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
    }
}
//javac exp5.java
//java exp5
```

Output:

```
Plain Text: Hello, Blowfish Encryption
Encrypted Text: HRAIgqc0fJNFNzKCASziU5q6UfcPcB7HoAp6HCexygM=
Decrypted Text: Hello, Blowfish Encryption
```

Experiment 6

Question:

Write the RC4 logic in Java Using Java cryptography; encrypt the text “Hello world” using Blowfish. Create your own key using Java key tool.

Code (exp6.java):

```
public class exp6
{
    private int[] S = new int[256];
    private int x = 0;
    private int y = 0;

    public exp6(byte[] key)
    {
        initialize(key);
    }

    private void initialize(byte[] key)
    {
        for (int i = 0; i < 256; i++)
        {
            S[i] = i;
        }

        int j = 0;

        for (int i = 0; i < 256; i++)
        {
            j = (j + S[i] + (key[i % key.length] & 0xFF)) & 0xFF;

            int temp = S[i];
            S[i] = S[j];
            S[j] = temp;
        }
    }

    public byte[] encryptDecrypt(byte[] input)
    {
        byte[] result = new byte[input.length];

        for (int i = 0; i < input.length; i++)
        {
            x = (x + 1) & 0xFF;
            y = (y + S[x]) & 0xFF;

            int temp = S[x];
            S[x] = S[y];
            S[y] = temp;

            int xorIndex = (S[x] + S[y]) & 0xFF;

            result[i] = (byte)(input[i] ^ S[xorIndex]);
        }

        return result;
    }

    public static void main(String[] args)
    {
        String text = "Hello World";

        byte[] key = "secretkey".getBytes();

        exp6 rc4 = new exp6(key);
```

```
        byte[] encrypted =
                rc4.encryptDecrypt(text.getBytes());

        System.out.println("Encrypted (Hex): "
                + bytesToHex(encrypted));

        exp6 rc4Decrypt = new exp6(key);

        byte[] decrypted =
                rc4Decrypt.encryptDecrypt(encrypted);

        System.out.println("Decrypted: "
                + new String(decrypted));
    }

    private static String bytesToHex(byte[] bytes)
    {
        StringBuilder sb = new StringBuilder();

        for (byte b : bytes)
        {
            sb.append(String.format("%02x", b & 0xFF));
        }

        return sb.toString();
    }
}

// Commands: javac exp6.java
// Commands: java exp6
```

Output:

```
Encrypted (Hex): edcdb0702391c2275ac02f
Decrypted: Hello World
```

Experiment 7

Question:

Write a Java program to implement RSA algorithm.

Code (exp7.java):

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PrivateKey;
import java.security.PublicKey;
import java.util.Base64;
import javax.crypto.Cipher;

public class exp7
{
    public static KeyPair generateRSAKeyPair(int keySize) throws Exception
    {
        KeyPairGenerator keyGen =
                KeyPairGenerator.getInstance("RSA");

        keyGen.initialize(keySize);

        return keyGen.generateKeyPair();
    }

    public static String encrypt(String plainText,
                                 PublicKey publicKey)
                                 throws Exception
    {
        Cipher encryptCipher =
                Cipher.getInstance("RSA");

        encryptCipher.init(
                Cipher.ENCRYPT_MODE,
                publicKey
        );

        byte[] encryptedBytes =
                encryptCipher.doFinal(
                        plainText.getBytes("UTF-8")
                );

        return Base64.getEncoder()
                .encodeToString(encryptedBytes);
    }

    public static String decrypt(String encryptedText,
                                  PrivateKey privateKey)
                                  throws Exception
    {
        Cipher decryptCipher =
                Cipher.getInstance("RSA");

        decryptCipher.init(
                Cipher.DECRYPT_MODE,
                privateKey
        );

        byte[] decryptedBytes =
                decryptCipher.doFinal(
                        Base64.getDecoder()
                        .decode(encryptedText)
                );

        return new String(
                decryptedBytes,
```

```
                "UTF-8"
        );
    }

    public static void main(String[] args)
    {
        try
        {
            KeyPair keyPair =
                    generateRSAKeyPair(2048);

            PublicKey publicKey =
                    keyPair.getPublic();

            PrivateKey privateKey =
                    keyPair.getPrivate();

            String plainText =
                    "Hello, RSA Encryption!";

            System.out.println(
                    "Plain Text: " + plainText
            );

            String originalText =
                    "Hello, RSA Encryption!";

            String encryptedText =
                    encrypt(originalText, publicKey);

            System.out.println(
                    "Encrypted Text: "
                    + encryptedText
            );

            String decryptedText =
                    decrypt(encryptedText, privateKey);

            System.out.println(
                    "Decrypted Text: "
                    + decryptedText
            );
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
    }
}

// Commands:
// javac exp7.java
// java exp7
```

Output:

```
Plain Text: Hello, RSA Encryption!
Encrypted Text: ck8yLfxOYdX+YC+EwJAvrXirzAq1Y+i+LV9sucmtvJTbAQ+Gyfrq33YtOAJk8WYuRRZvwmCq0b
GJ2+H/wNnpMAPG7KOoj7M2eD7q82hC7dXRoju6ipHIcm4F+TFYdu/+Ckyt1gmwdX0HGIVBOpHtbplJKQQ9ALkhiCYS
f3127dyUSmqydGEQy3msuTk/2jx4La+E38rfvdirgo+kn3xLsbJcgap//gmECHqvevtzMUP3+igfA37uNnNMqooLG8
cNUsfIxflG9B5+ww6VktTVeX02Qb9rC9Hal+RRA1uluUsn6ylPUQgR63Bkuutv9PpiCE74b5NcAQmjaSqmbVwdaQ==
Decrypted Text: Hello, RSA Encryption!
```

Experiment 8

Question:

Implement the Diffie-Hellman Key Exchange mechanism.

Code (exp8.java):

```
import java.security.*;
import java.util.Base64;
import javax.crypto.KeyAgreement;
import javax.crypto.spec.DHParameterSpec;

public class exp8
{
    public static void main(String[] args) throws Exception
    {
        // Generate Diffie-Hellman parameters
        AlgorithmParameterGenerator paramGen =
                AlgorithmParameterGenerator.getInstance("DH");

        paramGen.init(512);

        AlgorithmParameters params =
                paramGen.generateParameters();

        DHParameterSpec dhSpec =
                params.getParameterSpec(DHParameterSpec.class);

        // Generate key pair for Party A
        KeyPairGenerator keyPairGenA =
                KeyPairGenerator.getInstance("DH");

        keyPairGenA.initialize(dhSpec);

        KeyPair keyPairA =
                keyPairGenA.generateKeyPair();

        // Generate key pair for Party B
        KeyPairGenerator keyPairGenB =
                KeyPairGenerator.getInstance("DH");

        keyPairGenB.initialize(dhSpec);

        KeyPair keyPairB =
                keyPairGenB.generateKeyPair();

        // Key agreement for Party A
        KeyAgreement keyAgreeA =
                KeyAgreement.getInstance("DH");

        keyAgreeA.init(keyPairA.getPrivate());

        // Key agreement for Party B
        KeyAgreement keyAgreeB =
                KeyAgreement.getInstance("DH");

        keyAgreeB.init(keyPairB.getPrivate());

        // Generate shared secret
        keyAgreeA.doPhase(
                keyPairB.getPublic(), true);

        byte[] sharedSecretA =
                keyAgreeA.generateSecret();

        keyAgreeB.doPhase(
                keyPairA.getPublic(), true);
```

```
        byte[] sharedSecretB =
                keyAgreeB.generateSecret();

        // Display shared secrets
        System.out.println(
                "Party A Shared Secret: "
                + Base64.getEncoder()
                        .encodeToString(sharedSecretA)
        );

        System.out.println(
                "Party B Shared Secret: "
                + Base64.getEncoder()
                        .encodeToString(sharedSecretB)
        );

        // Check whether both secrets are equal
        System.out.println(
                "Keys match: "
                + MessageDigest.isEqual(
                        sharedSecretA,
                        sharedSecretB
                )
        );
    }
}

// Commands:
// javac exp8.java
// java exp8
```

Output:

```
Party A Shared Secret:
fZX7hl7enbElqKXBj+tZZr+IknW8+LBXTlV7+4zlZl+scWD2zIJREC/LGxk3aCtypn4eClmQkEU4i2KBcIaHjQ==
Party B Shared Secret:
fZX7hl7enbElqKXBj+tZZr+IknW8+LBXTlV7+4zlZl+scWD2zIJREC/LGxk3aCtypn4eClmQkEU4i2KBcIaHjQ==
Keys match: true
```

Experiment 9

Question:

Calculate the message digest of a text using the SHA-1 algorithm in JAVA.

Code (exp9.java):

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.Scanner;

public class exp9{

    public static String bytesToHex(byte[] bytes) {
        StringBuilder hexString = new StringBuilder();

        for (byte b : bytes) {
            String hex = Integer.toHexString(0xff & b);

            if (hex.length() == 1)
                hexString.append('0');

            hexString.append(hex);
        }

        return hexString.toString();
    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter text to hash using SHA-1: ");
        String input = scanner.nextLine();

        try {
            MessageDigest md = MessageDigest.getInstance("SHA-1");

            md.update(input.getBytes());

            byte[] digestBytes = md.digest();

            String hash = bytesToHex(digestBytes);

            System.out.println("SHA-1 hash of \"" + input + "\" is:");
            System.out.println(hash);

        } catch (NoSuchAlgorithmException e) {
            System.err.println("SHA-1 algorithm not found: " + e.getMessage());
        }

        scanner.close();
    }
}
//javac exp9.java
// java exp9
```

Output:

```
Enter text to hash using SHA-1: Hello World
 SHA-1 hash of "Hello World" is:
0a4d55a8d778e5022fab701977c5d840bbc486d0
```

Experiment 10

Question:

Calculate the message digest of a text using the MD5 algorithm in JAVA.

Code (exp10.java):

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.Scanner;

public class exp10{

    // Method to convert byte array to hex string
    public static String bytesToHex(byte[] bytes) {
        StringBuilder hexString = new StringBuilder();
        for (byte b : bytes) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1)
                hexString.append('0');
            hexString.append(hex);
        }
        return hexString.toString();
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter text to hash using MD5: ");
        String input = scanner.nextLine();

        try {
            // Create MD5 MessageDigest instance
            MessageDigest md = MessageDigest.getInstance("MD5");

            // Compute the digest
            byte[] digestBytes = md.digest(input.getBytes());

            // Convert the digest to hex string
            String hash = bytesToHex(digestBytes);

            // Print the result
            System.out.println("MD5 Hash: " + hash);

        } catch (NoSuchAlgorithmException e) {
            System.err.println("MD5 algorithm not found: " + e.getMessage());
        }

        scanner.close();
    }
}
//javac exp10.java
// java exp10
```

Output:

```
Enter text to hash using MD5: Hello World
 MD5 Hash: b10a8db164e0754105b7a99be72e3fe5
```